

CS301
DATA STRUCTURE AND ALGORITHMS
LECTURE 3: ADT & ARRAYS

Pandav Patel
Assistant Professor

Computer Engineering Department
Dharmsinh Desai University
Nadiad, Gujarat, India

OBJECTIVE

- Understand what is Abstract Data Type (ADT)
- Understand Array as an ADT
- Understand how to represent polynomial using an array

WHAT IS AN ABSTRACT DATA TYPE (ADT)?

- A set of **data values** and **associated operations** that are precisely specified independent of any particular implementation (source: [here](#))
- ADT defines only the interface of the operations; bereft of any implementation details
- Do you need to know internal working of a car in order to drive it?
- Do you need to know what happens to the engine when you
 - start it
 - press clutch
 - change gear
 - press accelerator
- No, because a car has well defined interface (abstraction)
- If you needed to know the internal details of a car in order to drive it; then you would have to learn to drive each model separately

MINIMUM REQUIRED FUNCTIONALITY (MRF)

- An ADT specifies Minimum Required Functionality (MRF)
- MRF defines minimum operations which should be supported by an implementation of given ADT
- An implementation may support operations in addition to the operations specified in MRF

EXAMPLE: ARRAY AS AN ADT

- MRF for an Array (ADT)
 - $get(i)$ - returns value at index i
 - $set(i, v)$ - set value v at index i
- We can not talk about complexities of get and set operations, as they depend on implementation
- An implementation may provide additional operations (not part of the ADT)
 - $size()$ - returns size of an array i
 - $resize(n)$ - resize the array such that it can hold n values
 - $at(i)$ - returns value at index i if i is a valid index.
 - $check(x)$ - returns *true* if x is present in the array, otherwise returns *false*
 - $print()$ - print all the values stored in an array
 - $find(x)$ - returns index of first occurrence of x , if x is present

ARRAY (DATA STRUCTURE)

- For most implementations of array (ADT)
 - Values are of the same type (homogeneous)
 - Memory is allocated contiguously
 - So *get* and *set* can be performed in constant time
- For C like languages
 - *get* and *set* can be done using []
 - $v = arr[i];$
 - $arr[i] = v;$
- According to our definition of Array (ADT) in previous slide, *vector* in C++ is also an array.
 - C++ *vector* supports some additional operations
 - e.g. *size()*, *at(i)*

ADT Vs DATA STRUCTURE

- ADT only provides interface (it is logical description)
- Data structure concerns with actual representation of data (implementation details)

REPRESENT POLYNOMIAL USING AN ARRAY

- Polynomial examples
 - $A(x) = x^2 + 3x + 4$
 - $B(x) = 3x^2 + 5x + 7$
- How to represent $A(x)$ and $B(x)$ using an array?
 - What should be the size of the array?
 - What should be the type of the elements?
 - Think of representing $x^{200} + 7$
 - What is the problem?
- Can you think of any alternate representation using an array?
 - An array of objects? What would be the structure of each object?
 - Do you need to know maximum power or number of terms in advance?
- How would you add or multiply polynomials in each of those representations?

REPRESENT POLYNOMIAL USING AN ARRAY (CONT...)

SOLUTIONS

■ Approach I

```
const int MAXDEGREE = 10;
int degree;
float coefficients[MAXDEGREE + 1];
```

■ Approach II

```
int degree;
cin >> degree;
float coefficients[degree + 1];
```

REPRESENT POLYNOMIAL USING AN ARRAY (CONT...)

SOLUTIONS

■ Approach III

```
struct term {  
    int power;  
    float coefficient;  
};  
int maxterms;  
cin >> maxterms;  
struct term terms[maxterms];
```

- Any other approach???
- THINK: How would you add or multiply two polynomials for each of the above approaches.
- THINK: What are the drawbacks of each of these approaches?

